1 Introduction and Overview
2 Visual Search System
3 Google Photo Text Querying System
4 YouTube Video Search
5 Hashtag Classification System
6 Video Recommendation System
7 Ad CTR Prediction and Ranking System
8 Ad Click Prediction on Social Platforms
9 Similar Listings on Vacation Rental Platform
10 Personalized News Feed
11 Nearby Places

04
YouTube Video Search

Our video sharing platforms such as YouTube, the number of videos uploaded per minute keeps growing. In this chapter, we design a video search system that can effectively find the relevant videos for a given text query. We discuss a high-level design for Figure 4.1 that can serve both the text-to-video retrieval and video-to-video retrieval. We also briefly discuss the most relevant videos for the given text.

Figure 4.1: Searching videos with a text query.

Clarifying Requirements

Here is a typical interaction between a candidate and an interviewer.

Candidate: In the input query, text only, or can users search with an image or video?
Interviewer: Text only.
Candidate: Do we want to personalize the video search results? Should we consider user history for different users, based on their preferences?
Interviewer: No, exposure to recommendation systems where personalization is essential, as do not necessarily have to personalize results in search platforms. We want to evaluate the model's ability to understand videos and their content.
Candidate: Do we want to personalize the search results for a given user, based on their location?
Interviewer: For simplicity, let's assume only English is supported.
Candidate: How many videos are available on the platform?
Interviewer: Billions of videos.
Candidate: Do we need to personalize the results? Should we rank the results differently for different users, based on their interactions?
Interviewer: As exposure to recommendation systems where personalization is essential, we do not necessarily have to personalize results in search platforms. We want to evaluate the model's ability to understand videos and their content.

Figure 4.2: Video search system's input output

Frame the Problem as an ML Task

Defining the ML objective

Video search system is expected to provide relevant and useful results. One way to formulate the ML objective is to rank videos based on their relevance to the text query.

Specifying the system's input and output

As shown in Figure 4.2, the search system takes a text query as input and outputs a ranked list of videos sorted by their relevance to the text query.

Figure 4.3 High level overview of the search system

Choosing the right ML category

In order to determine the relevance between a video and a query, we utilize both visual content and the similarity between the embeddings of the video and the query. We will cover in Figure 4.4.

Figure 4.4 High level overview of the search system

Let's briefly discuss each component.

Visual search

This component takes a text query as input and outputs a list of videos. The search system calculates the similarity between the query and all videos in the database.

Text search

Figure 4.5 shows how text search works when a user types in a text query. "dogs playing indoor". Videos with the closest video embeddings, as depicted in Figure 4.6, are ranked higher in the position.

Figure 4.5 Text search system

Figure 4.6 ML model's input output

In order to rank videos that are visually and semantically similar to the text query, we compute the dot product between the text query embedding and video embeddings to rank the videos based on their similarity scores.

Text encoder

Figure 4.7 shows how text search works when a user types in a text query. "dogs playing indoor". Videos with the closest video embeddings, as depicted in Figure 4.8, are ranked higher in the position.

Figure 4.7 Text search system

	Title	Text
1	My dog is playing inside	Dog indoor play
2	The recording is performed in a quiet room with no background noise	Speech recorded quiet room
3	Lorem ipsum dolor sitting amet consectetur	Lorem ipsum dolor sit amet
4	Tip: spread seeds when planting	Tip seed plant
5	Tip: spread seeds when planting	Tip seed plant
6	Tip: spread seeds when planting	Tip seed plant
7	Lorem ipsum dolor sitting amet consectetur	Lorem ipsum dolor sit amet
8	Dogs playing outdoor	Dog outdoor play
9	Dogs playing indoor	Dog indoor play

Figure 4.8 Text search

The inverted index is a common technique for creating the text search component, allowing efficient full-text search in databases. Elasticsearch and Lucene indexes aren't based on machine learning, they're in training-free. A popular search engine companies use for their text search. Lucene is a readily-used engine and documented index. The inverted index will a deeper understanding of Elasticsearch's role. [1]

Data Preparation

Data engineering

Since we are given a dataset between text and produced the inverted index, it's not necessary to perform any data engineering. Table 4.1 shows what the annotated dataset might look like.

Video name	Source	Split type
78100.mp4	Dog surrounding a carpet	Training
52387.mp4	Celebrating graduation	Training
24811.mp4	A group of teenagers playing soccer	Validation
69854.mp4	Video Thumbnail before	Validation
75354.mp4	Road trip in winter	Test

Table 4.1. Annotated dataset

Feature engineering

Almost all ML algorithms accept only numeric input values. Unstructured data such as texts and videos need to be converted into a numerical representation during the step. Let's take a look at how to prepare text and video data for the model.

Preparing text data

As shown in Figure 4.9, text is typically represented as a numerical vector using three steps: text normalization, tokenization, and tokens to IDs. [2]

Figure 4.9: Preprocess a text data in a numerical vector

Let's take a look at each step in more detail.

Text normalization

Text normalization - also known as text cleanup - prepares words and sentences as consistent. For example, the same word may be spelled slightly differently, such as "hug", "dogs", and "DOGS" all refer to the same thing but are spelled in different ways. The same is true for sentences. Take these two sentences, for example:

- "A person is walking on Wednesday"
- "A person walking on a number"

Both sentences mean the same, but have differing punctuation and verb forms. Here are some typical text normalization techniques:

- Lowercasing: Ignore all letters lowercase, as this does not change the meaning of words or sentences.
- Contractions expansion: Convert contractions into their full forms. Common punctuation marks are exclamation, comma, apostrophe, period.
- Stemming and lemmatization: Convert words into their root forms. Stemming is a combination of simple rules.
- Stop words: remove stop words from the text. For example, Stopwords = ["a", "an", "the", "is", "are", "at", "by", "for", "from"]
- Lowercasing and stemming: Identify a canonical representative for a set of related word forms. For example, walking, walks, walked → walk.

Tokenization

Tokenization is the process of breaking down a piece of text into smaller units called tokens. Generally, there are three types of tokenization:

- Word tokenization: split the text into individual words based on specific delimiters. For example, a phrase like "I have an interesting vacation" becomes ["I", "have", "an", "interesting", "vacation"].
- Subword tokenization: split text into subwords (or n-grams) characters.
- Character tokenization: split text into a set of characters. The advantage of subword tokenization algorithms are usually a strong trade-off between vocabulary size and token length. For example, "Walking" → ["walk", "ing"].

Tokens to IDs

Once we have the tokens, we need to convert them to numerical representations. The representation of tokens with numerical values can be done in several ways:

- Lookup table
- Hashing

Lookup table. In this method, each unique token is mapped to an ID. Hence, a lookup table is created to store these 1 : 1 mappings. Figure 4.10 shows what the mapping table might look like.

Figure 4.10 Lookup table

Hashing. Hashing, also called "feature hashing" or "hashing trick", is a memory-efficient method that uses a hash function to obtain IDs, without keeping a lookup table. Figure 4.9 shows how a hash function is used to convert words to IDs.

Figure 4.11 Use hashing to obtain word IDs

Let's compare the lookup table with the hashing method.

Lookup table	Hashing
• Table is created mapping to IDs.	• Table is computed hash function to convert tokens to IDs.
• Table is created IDs to tokens using reverse index table.	• Not possible to convert IDs to tokens.
• The table is stored in memory. It slows down training and inference if the table is stored in a memory mapped.	• Hash function is sufficient to convert any token to ID.
• Token unseen words cannot be properly handled.	• Easily handles new or unseen words by applying the hash function to any word.
Collisions (IDs)	
• No collision issue.	• Collisions are a potential problem.

Table 4.2 Lookup table vs. feature hashing

Preparing video data

Figure 4.12 shows a typical workflow for preprocessing a raw video.

Figure 4.12 Video preprocessing pipeline

Model Development

Model selection

As discussed in the "Framing the problem as an ML task" section, dot-product are combined into embeddings by a two encoders. The two encoders combine embeddings by a dot-product. In this section, we compare multiple popular architectures for video encoder.

A typical text encoder's input and output are shown in Figure 4.13.

Figure 4.13. Text encoder's input output

The text encoder converts text into a vector representation [3]. For example, if two sentences have similar meanings, their embeddings are more similar. To build the text encoder, three popular architectures are available: statistical methods and BM-based methods, and transformer-based.

Statistical methods

Statistical methods rely on statistics to convert a sentence into a feature vector. Two popular statistical methods are:

- Bag of Words (BoW)
- Term Frequency-Inverse Document Frequency (TF-IDF)

BoW. This method converts a sentence into a fixed-length vector. It creates vocabulary-accompanied by creating a table with rows representing sentences, and columns representing each token. An example of BoW is shown in Figure 4.14.

	"walk"	"walking"	"to"	"store"	"yesterday"	"going"	"the"	"very"	"with"
this sentence is too very close	0	0	1	0	0	0	1	1	0
walking to the store	0	1	1	1	0	0	0	0	0
We ship with tested spare in bowl	0	0	0	0	0	0	0	0	1

Figure 4.14 BoW representations of different sentences

Advantages of BoW over other statistical methods for representations text are the following:

- It does not consider the order of words in a sentence. For example, "let's watch TV after work" and "let's work after watch TV" have the same representation, even though they differ.
- The inherent dependencies does not capture the semantic and contextual meaning of the sentence. For example, two sentences with the same meaning but different words have a totally different representation.
- The vocabulary size can be huge, leading to sparse representations if sentences are short. The number of unique tokens in the training corpus is usually large, so each sentence representation is mostly filled with zeros.

TF-IDF. This is a numerical statistic intended to reflect how important a word is to a document in a collection or corpus. TF-IDF covers the same sentence and token as in BoW, but it normalizes the hash based on the frequency of words. To learn more about the mathematics behind this, refer to [4].

The TF-IDF gives less weight to frequent words, its representations are usually better than BoW. However, it has the following drawbacks:

- It computation step is needed to decompose term frequencies when a new sentence is added.
- It does not consider the order of words in a sentence.
- The inherent dependencies does not capture the semantic meaning of the sentence.
- Vocabulary size can be huge.

In summary, statistical methods are quick and fast. However, they do not capture the contextual meaning of sentences, and the representations are sparse. We overcome these drawbacks using transformers.

BM-based methods

In these methods, all BM (word embedding) sentences into meaningful word embeddings so that the distance between two embeddings reflects the semantic similarity of the corresponding words. For example, if two words, cat and dog, are "similar" they are semantically similar, their embeddings are close in the embedding space. Figure 4.15 shows a simple visualization of word embeddings in 2-dimensional space. In real-world, words are in 256-dimensional space.

Figure 4.15 Words in the 2D embedding space

There are three common BM, based approaches for transforming texts into embeddings:

- Embedding lookup layer
- Word2vec
- Transformer-based architectures

Embedding lookup layer In this approach, an embedding layer is employed to map each ID to an embedding vector. Figure 4.16 shows an example.

Figure 4.16 Embedding layer

Embedding lookup layer is a simple and effective solution to convert sparse features, such as IDs, into a dense representation. Embedding layer can be trained together with other layers.

Word2vec. Word2vec [5] is a family of shallow models used to produce word embeddings. These models use a shallow neural network to produce a dense vector representation for each word in a corpus. Word2vec's pre-trained word embeddings, in particular, the Google News pre-trained vectors, are a boost factor to surmounting cold-start. Using the Keras package, this feature is quite straightforward. Word2vec's two popular architectures are CBOW and Skip-gram.

There are two other embedding techniques: Continuous Bag-of-Words (CBOW) [6] and Skip-gram [7]. Figure 4.17 shows how CBOW works at a high level. If you are interested to learn about these models, refer to [8].

Figure 4.17: CBOW model

Even though word2vec and embedding layers are simple and effective, recent contextual word representations have been shown promising results.

Transformer-based models

These models consider the context of the words in a sentence when converting them into embeddings. As opposed to word2vec, these models produce different embeddings for the same word depending on the context.

Figure 4.18 shows a Transformer-based model which takes a sentence, a set of tokens in the input, and produces an embedding for each token.

Figure 4.18 Transformer encoder

Transformers are very powerful at understanding the context and producing meaningful embeddings. Several models, such as BERT, have been pre-trained on large text corpora, such as Wikipedia and BooksCorpus, and have been used for transfer learning for many downstream natural language processing (NLP) tasks. Popular Transformer-based architecture such as BERT can be fine-tuned.

In some instances, the inter-frame may want you to dive deeper into the details of the Transformer-based models. To learn more, refer to [9].

Video encoder

We have two architectural options for encoding video. We have two architectural options for encoding video.

1. Video level models
2. Frame level models

Video level models process a whole video to create an embedding, as shown in Figure 4.19. The model architecture is usually based on 3D convolutions [10]. Transformers have the inherent power to model long-range video dependencies.

Figure 4.19 A video level model

Frame level models score of the frame level, it is often faster and computationally less expensive. However, frame level models require an aggregation layer to combine the embeddings from individual frames to produce a video level embedding. Simple but effective approach is to average frame level embeddings to create a video level embedding. We can also use a Transformer-based encoder to combine frame level embeddings to create a video level embedding.

Figure 4.20 A frame level model

Since this model centers on the frame level, it is often faster and computationally less expensive. However, frame level models require an aggregation layer to combine the embeddings from individual frames to produce a frame level embedding. Simple but effective approach is to average frame level embeddings to create a video level embedding. We can also use a Transformer-based encoder to combine frame level embeddings to create a video level embedding.

- Improve the training and serving speed
- Reduce the number of computations

Model training

To train the text encoder and video encoder, we use a contrastive learning approach. If you are interested in learning more about this, see the "Model training" section in Chapter 2, Visual Search System.

An explanation of how to compute the loss during model training is shown in Figure 4.18.

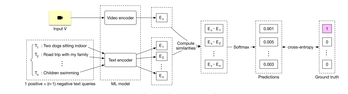

Figure 4.18: Loss computation

Evaluation

Offline metrics

Here are some offline metrics that are typically used in search systems. Let's examine which are the most relevant.

Precision@k and mAP

$$\text{precision@k} = \frac{\text{number of relevant items among the top } k \text{ items in the ranked list}}{k}$$

In the evaluation dataset, a given text query is associated with only one video. That means the numerator of the precision@k formula is at most 1. This leads to low precision@k values. For example, for a given text query, even if we rank its associated video at the top of the list, the precision@10 is only 0.1. Due to this limitation, precision metrics, such as precision@k and mAP, are not very helpful.

Recall@k. This measures the ratio between the number of relevant videos in the search results and the total number of relevant videos.

$$\text{recall@k} = \frac{\text{Number of relevant videos among the top } k \text{ videos}}{\text{Total number of relevant videos}}$$

As described earlier, the "total number of relevant videos" is always 1 . With that, we can translate the recall@k formula to the following:

recall@ k = 1 if the relevant video is among the top k videos, 0 otherwise

What are the pros and cons of this metric?

Pros

- It effectively measures a model's ability to find the associated video for a given text query.

Cons

- It depends on k. Choosing the right k could be challenging.
- When the relevant video is not among the k videos in the output list, recall@k is always 0. For example, consider the case where model A ranks a relevant video at place 15, and model B ranks the same video at place 50. If we use recall@10 to measure the quality of these two models, both would have recall@10=0, even though model A is better than model B.

Mean Reciprocal Rank (MRR). This metric measures the quality of the model by averaging the rank of the first relevant item in each search result. The formula is:

$$MRR = \frac{1}{m}\sum_{i=1}^{m}\frac{1}{rank_i}$$

This metric addresses the shortcomings of recall@k and can be used as our offline metric.

Online metrics

As part of online evaluation, companies track a wide variety of metrics. Let's take a look at some of the most important ones:

- Click-through rate (CTR)
- Video completion rate
- Total watch time of search results

CTR. This metric shows how often users click on retrieved videos. The main problem with CTR is that it does not track whether the clicked videos are relevant to the user. In spite of this issue, CTR is still a good metric to track because it shows how many people clicked on search results.

Video completion rate. A metric measuring how many videos appear in search results and are watched by users until the end. The problem with this metric is that a user may watch a video only partially, but still find it relevant. The video completion rate alone cannot reflect the relevance of search results.

Total watch time of search results. This metric tracks the total time users spent watching the videos returned by the search results. Users tend to spend more time watching if the search results are relevant. This metric is a good indication of how relevant the search results are.

Serving

At serving time, the system displays a ranked list of videos relevant to a given text query. Figure **4.19** shows a simplified ML system design.

Figure 4.19: ML system design

Let's discuss each pipeline in more detail.

Prediction pipeline

This pipeline consists of:

- Visual search
- Text search
- Fusing layer
- Re-ranking service

Visual search. This component encodes the text query and uses the nearest neighbor service to find the most similar video embeddings to the text embedding. To accelerate the NN search, we use approximate nearest neighbor (ANN) algorithms, as described in Chapter 2, Visual Search System.

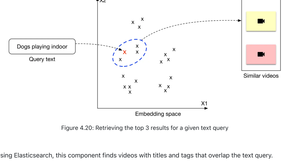

Figure 4.20: Retrieving the top 3 results for a given text query

Text search. Using Elasticsearch, this component finds videos with titles and tags that overlap the text query.

Fusing layer. This component takes two different lists of relevant videos from the previous step, and combines them into a new list of videos.

The fusing layer can be implemented in two ways, the easiest of which is to re-rank videos based on the weighted sum of their predicted relevance scores. A more complex approach is to adapt an additional model to re-rank the videos, which is more expensive because it requires model training. Additionally, it's slower at serving. As a result, we use the former approach.

Re-ranking service. This service modifies the ranked list of videos by incorporating business-level logic and policies.

Video indexing pipeline

A trained video encoder is used to compute video embeddings, which are then indexed. These indexed video embeddings are used by the nearest neighbor service.

Text indexing pipeline

This uses Elasticsearch for indexing titles, manual tags, and auto-generated tags.

Usually, when a user uploads a video, they provide tags to help better identify the video. But what if they do not manually enter tags? One option is to use a standalone model to generate tags. We name this component the auto-tagger and it is especially valuable in cases where a video has no manual tags. These tags may be noisier than manual tags, but they are still valuable.

Other Talking Points

Before concluding this chapter, it's important to note we have simplified the system design of the video search system. In practice, it is much more complex. Some improvements may include:

- Use a multi-stage design (candidate generation + ranking).
- Use more video features such as video length, video popularity, etc.
- Instead of relying on annotated data, use interactions (e.g., clicks, likes, etc.) to construct and label data. This allows us to continuously train the model.
- Use an ML model to find titles and tags which are semantically similar to the text query. This model can be combined with Elasticsearch to improve search quality.

If there's time left at the end of the interview, here are some additional talking points:

- An important topic in search systems is query understanding, such as spelling correction, query category identification, and entity recognition. How to build a queryunderstanding component? [17].
- How to build a multi-modal system that processes speech and audio to improve search results [18].
- How to extend this work to support other languages [19].
- Near-duplicate videos in the final output may negatively impact user experience. How to detect near-duplicate videos so we can remove them before displaying the results [20] ?
- Text queries can be divided into head, torso, and tail queries. What are the different approaches commonly used in each case [21]?
- How to consider popularity and freshness when producing the output list [22]?
- How real-world search systems work [23][24][25].

References

1. Elasticsearch. https://www.tutorialspoint.com/elasticsearch/elasticsearch_query_dsl.htm.
2. Preprocessing text data. https://huggingface.co/docs/transformers/v4.42.0/preprocessing.
3. NFKD normalization. https://unicode.org/reports/tr15/.
4. What is Tokenization summary. https://huggingface.co/docs/transformers/tokenizer_summary.
5. Hash collision. https://en.wikipedia.org/wiki/Hash_collision.
6. Deep learning for NLP. http://cs224d.stanford.edu/lecture_notes/notes1.pdf.
7. TF-IDF. https://en.wikipedia.org/wiki/TF%E2%80%93IDF.
8. Word2Vec models. https://www.tensorflow.org/tutorials/text/word2vec.
9. Continuous bag of words. https://www.kdnuggets.com/2018/04/implementing-deep-learning-methods-feature-engineering-text-data-cbow.html.
10. Skip-gram model. http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/.
11. BERT model. https://arxiv.org/pdf/1810.04805.pdf.
12. GPT3 model. https://arxiv.org/pdf/2005.14165.pdf.
13. BLOOM model. https://bigscience.huggingface.co/blog/bloom.
14. Transformer implementation from scratch. https://peterbloem.nl/blog/transformers.
15. 3D convolutions. https://www.kaggle.com/code/shivamb/3d-convolutions-understanding-use-case/notebook.
16. Vision Transformer. https://arxiv.org/pdf/2010.11929.pdf.
17. Query understanding for search engines. https://www.linkedin.com/pulse/ai-query-understanding-daniel-tunkelang/.
18. Multimodal video representation learning. https://arxiv.org/pdf/2012.04124.pdf.
19. Multilingual language models. https://arxiv.org/pdf/2107.00676.pdf.
20. Near-duplicate video detection. https://arxiv.org/pdf/2005.07356.pdf.
21. Generalizable search relevance. https://livebook.manning.com/book/ai-powered-search/chapter-10/v-10/20.
22. Freshness in search and recommendation systems. https://developers.google.com/machine-learning/recommendation/dnn/re-ranking.
23. Semantic product search by Amazon. https://arxiv.org/pdf/1907.00937.pdf.
24. Ranking relevance in Yahoo search. https://www.kdd.org/kdd2016/papers/files/adf0361-yinA.pdf.
25. Semantic product search in E-Commerce. https://arxiv.org/pdf/2008.08180.pdf.

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask AI

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.